

Sendra, service par service

Microservices FastAPI totalement découplés, monolithe Django modulaire, ou hybride — comparés point par point sur les quinze critères demandés, avec les douze schémas d'architecture.

A — Microservices FastAPI

B — Django modulaire

C — Hybride séquencé — **recommandé**

DÉCISION DU CHEF DE PROJET

Le **Scénario A** est retenu et maintenu tel quel. Ce document reste en l'état comme référence des compromis assumés en choisissant le découplage complet — et se complète ci-dessous de la couche DevOps, du service de journalisation des activités, et des services additionnels à considérer.

00 Les trois scénarios, en bref

SCÉNARIO A

Microservices FastAPI

- 1 service = 1 responsabilité = 1 base PostgreSQL
- Keycloak pour l'identité, Kafka pour les événements
- MinIO, Novu, OpenSearch, Redis en composants partagés
- Déploiement indépendant par service

Prématuré à ce stade

SCÉNARIO B

Django modulaire

- Un monolithe, des domaines cloisonnés en apps Django
- Une base PostgreSQL, Redis + Celery pour l'asynchrone
- MinIO centralisé, OpenSearch en option
- Un seul cycle de déploiement

Solide, mais tout ou rien

SCÉNARIO C

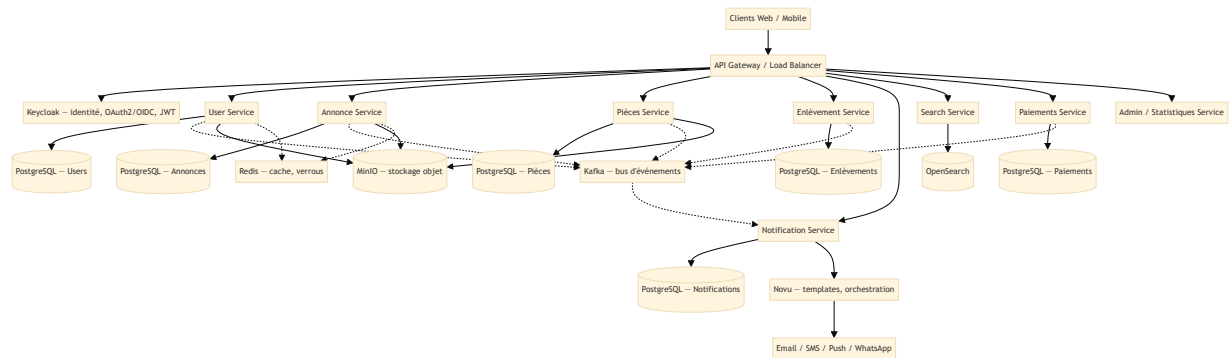
Hybride séquentiel

- Cœur métier en monolithe modulaire Django
- Notifications et Recherche extraits dès le départ
- Kafka et Keycloak activés seulement si un besoin réel apparaît
- Chaque domaine reste extractible plus tard, sans réécriture

Recommandé

01 Architecture microservices FastAPI (Scénario A)

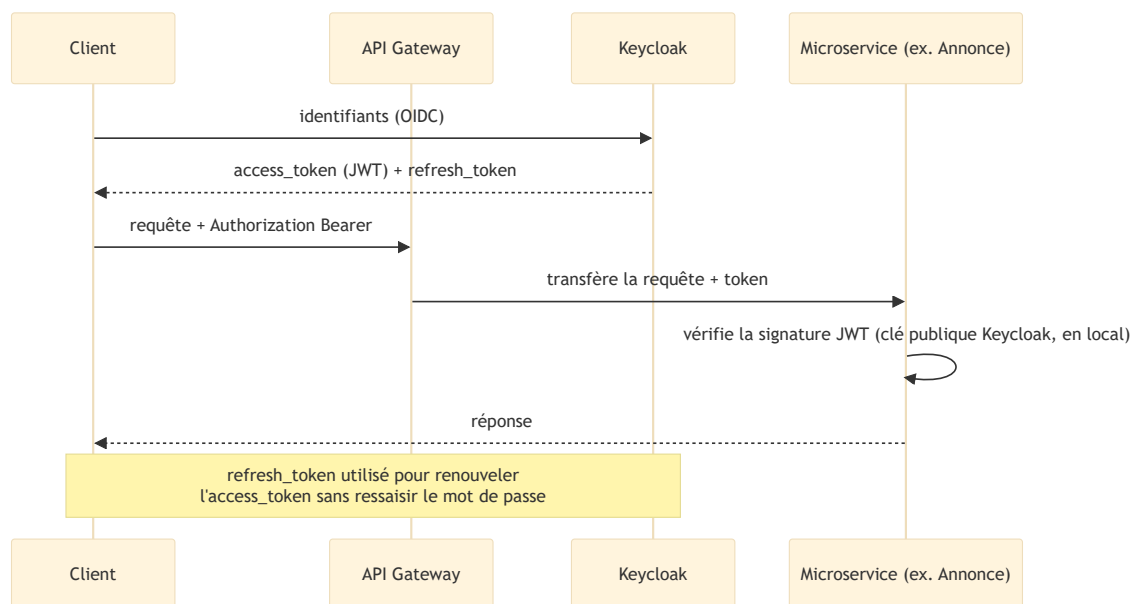
DÉCOUPAGE PAR DOMAINE, UNE BASE POSTGRESQL PAR SERVICE



Huit services, huit bases, plus Keycloak/Kafka/MinIO/Novu/OpenSearch/Redis en composants d'infrastructure. Onze systèmes distincts à faire fonctionner ensemble, chacun avec sa propre disponibilité à garantir.

02 Flux d'authentification — Keycloak

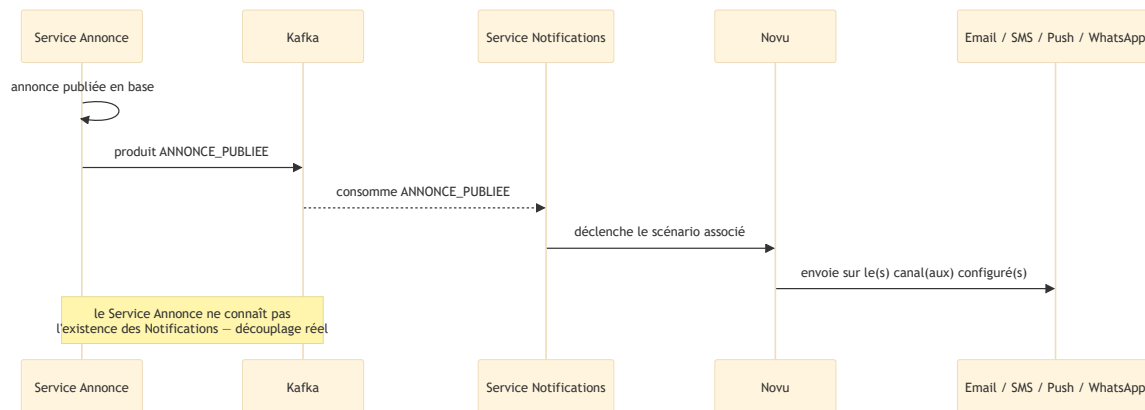
OAuth2 / OpenID Connect, VÉRIFICATION JWT CÔTÉ SERVICE



Chaque service vérifie le JWT localement (clé publique Keycloak mise en cache) — pas d'appel réseau à Keycloak à chaque requête. C'est le point fort réel de Keycloak : une fois en place, il ne devient pas un goulot d'étranglement.

03 Événements Kafka et service Notifications

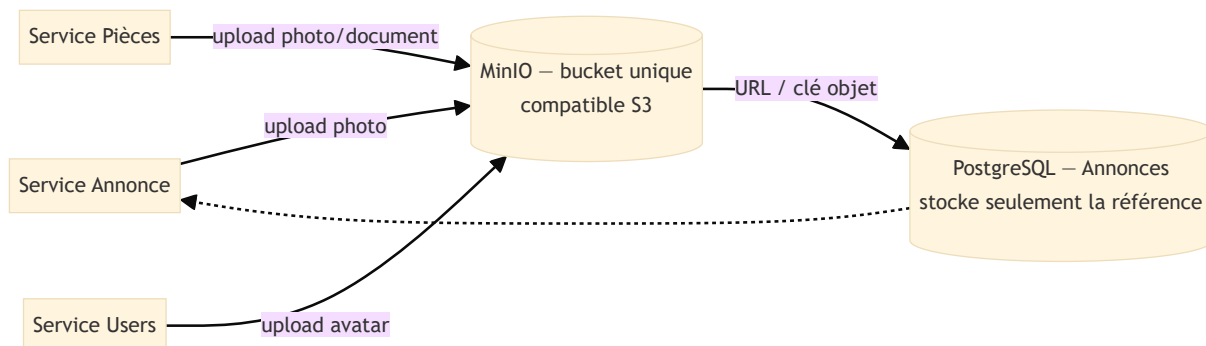
SERVICE ANNONCE → KAFKA → SERVICE NOTIFICATIONS → NOVU → CANAUX



Le découplage fonctionne réellement ici : le service producteur (Annonce) ignore tout du destinataire de l'événement. C'est la partie la plus solide du Scénario A — et elle ne nécessite pas d'adopter le reste de l'architecture pour en profiter (voir Scénario C).

04 MinIO — stockage objet centralisé

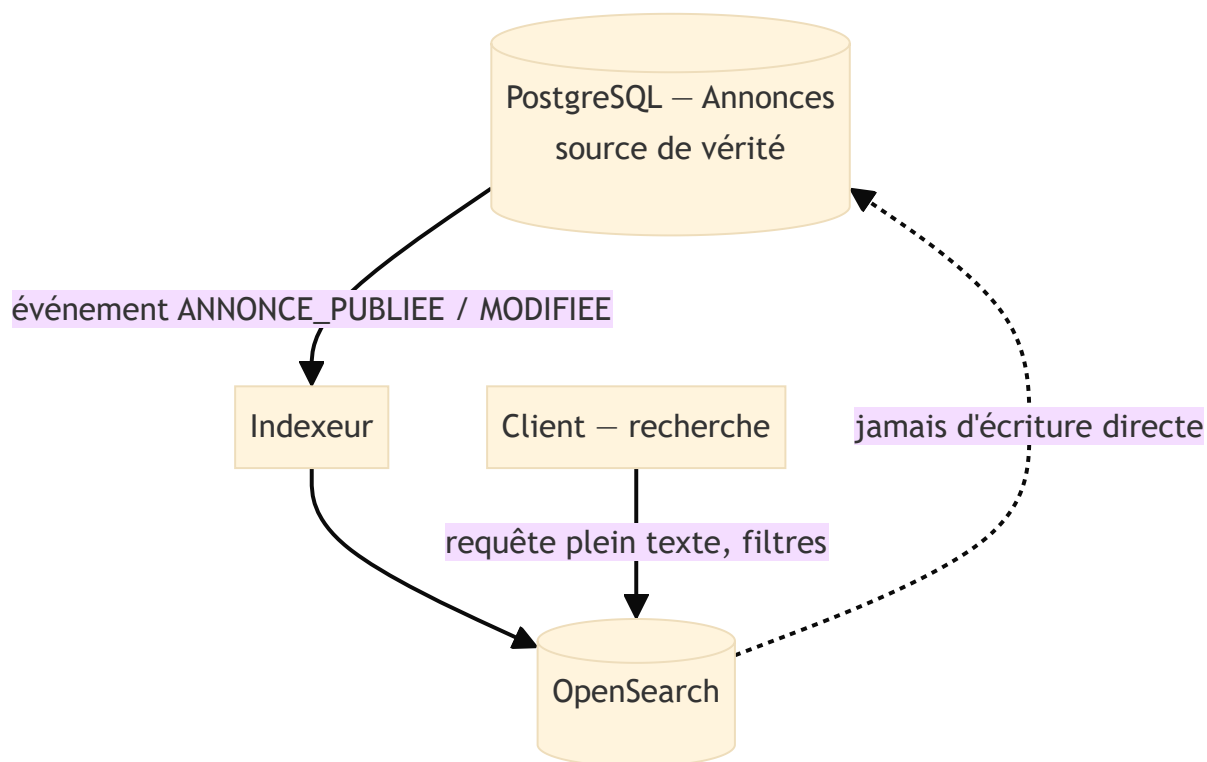
CHAQUE SERVICE ÉCRIT L'OBJET, NE GARDE QUE LA RÉFÉRENCE EN BASE



C'est le composant le plus consensuel de toute l'étude — utile dans **les trois** scénarios, y compris le monolithe Django. C'est aussi la correction directe du bug CORS constaté cette semaine sur Sendra actuel (fichiers sur le disque d'un seul serveur, référencés depuis un autre domaine).

05 OpenSearch — synchronisation

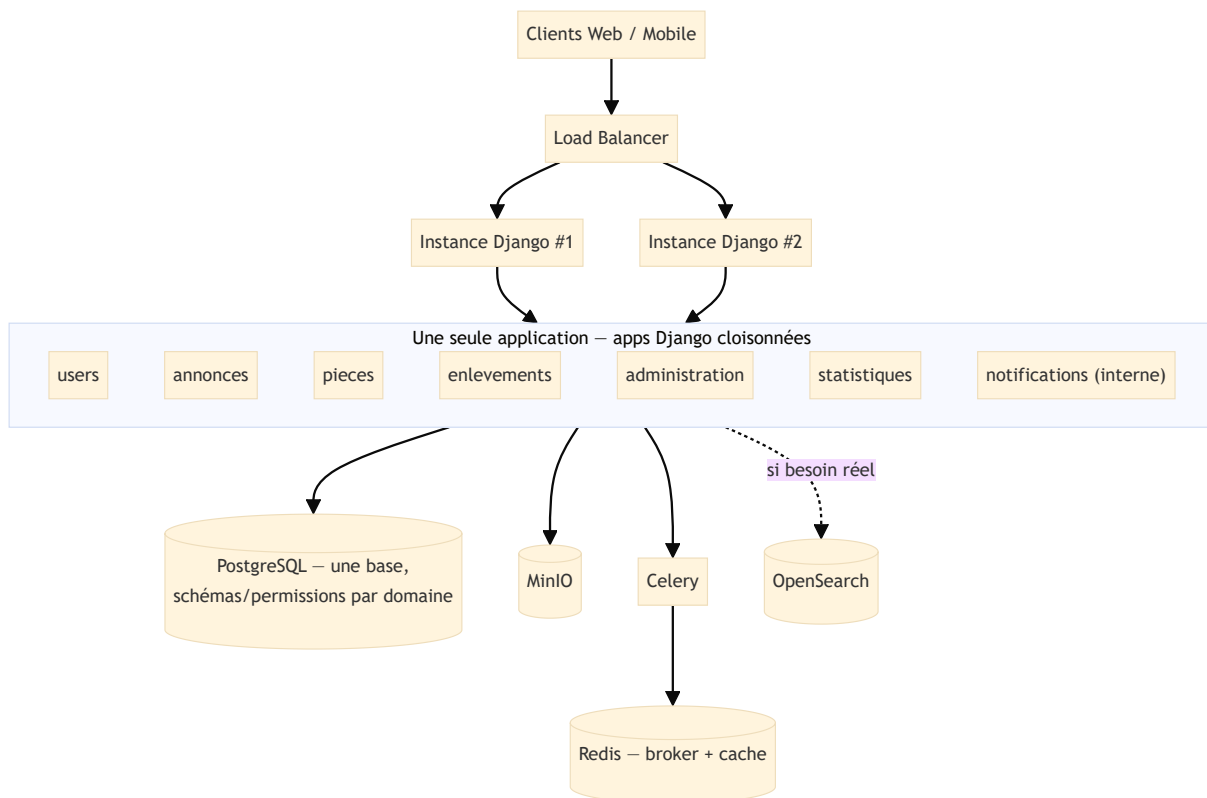
INDEX EN LECTURE SEULE, JAMAIS SOURCE DE VÉRITÉ



OpenSearch reste une projection dérivée, jamais la base d'écriture — un décalage de quelques secondes entre publication et recherche est acceptable, ce qui simplifie beaucoup la synchronisation par rapport à une cohérence stricte.

06 Architecture Django alternative (Scénario B)

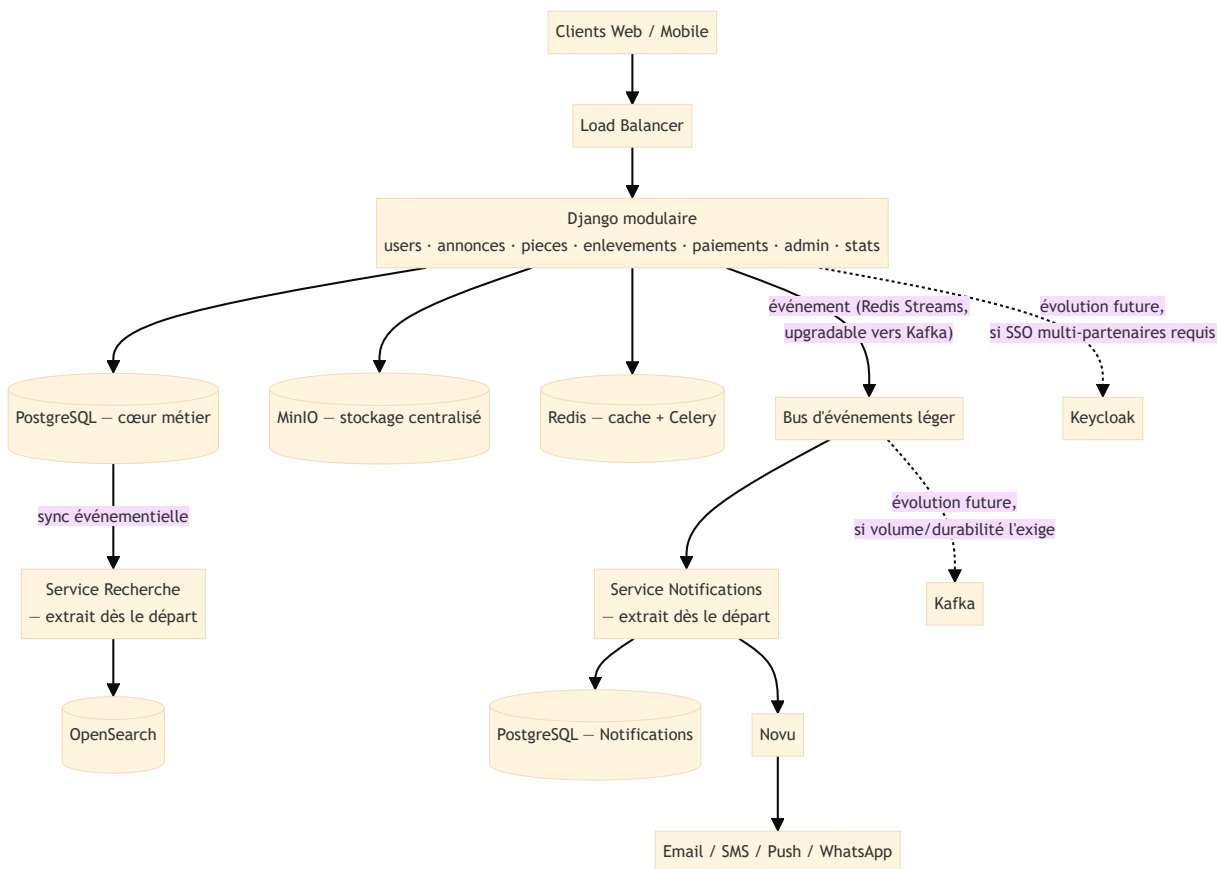
UN MONOLITHE, DES DOMAINES CLOISONNÉS EN APPS INTERNES



Les domaines restent cloisonnés au niveau du code (apps Django, permissions, éventuellement des schémas PostgreSQL séparés) sans payer le coût réseau et transactionnel de la séparation physique en services.

07 Architecture hybride recommandée (Scénario C)

LE CŒUR MÉTIER RESTE UNI ; SEULS LES DOMAINES QUI EN TIRENT UN VRAI BÉNÉFICE SONT EXTRAITS

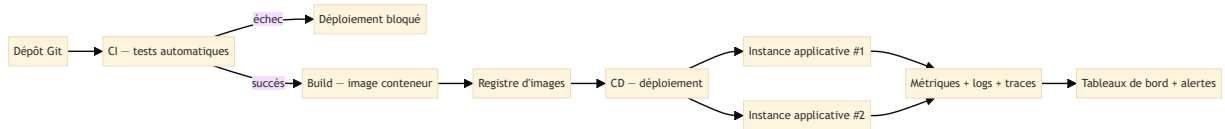


Notifications et Recherche sont extraits *dès maintenant* – ce sont des projections dérivées, jamais source de vérité transactionnelle, donc sûres à découpler sans complexité de transaction distribuée.

Paiements/Pièces/Enlèvements restent ensemble tant qu'ils partagent une même transaction métier (publier une pièce → l'enlever → la payer est un seul parcours cohérent).

08 Déploiement scalable, DevOps et observabilité

VALABLE POUR LES TROIS SCÉNARIOS — C'EST LA PARTIE QUI MANQUE AUJOURD'HUI À SENDRA, QUEL QUE SOIT LE CHOIX RETENU



Le pipeline CI/CD et l'observabilité (métriques, logs, traces) sont indépendants du choix d'architecture — et c'est précisément ce qui manque à Sendra aujourd'hui (déploiement manuel en SSH, aucun test bloquant). À construire en premier, quel que soit le scénario retenu ensuite.

09 Comparatif, critère par critère

Les quinze critères demandés, pour les trois scénarios.

CRITÈRE	A — MICROSERVICES FASTAPI	B — DJANGO MODULAIRE	C —
Scalabilité	Chaque service scale indépendamment — un vrai atout, mais seulement utile si un service précis a un profil de charge très différent des autres.	Scale horizontalement en dupliquant l'instance complète — suffisant jusqu'à plusieurs centaines de milliers d'utilisateurs si la base et le cache sont bien dimensionnés.	Le No ind rée rec
Performances	FastAPI/async excellent en I/O pur, mais la latence inter-services (réseau, sérialisation) s'ajoute à chaque appel traversant plusieurs domaines.	Aucune latence réseau interne — un appel de fonction Python, pas une requête HTTP entre services.	Lat des late dev (év
Complexité de développement	Élevée — schémas d'événements à versionner, contrats d'API entre services, tests d'intégration multi-services.	Modérée — la discipline de cloisonnement (apps, permissions) doit être tenue manuellement, sans barrière physique.	Mo ser cor rap
Coût d'infrastructure	Élevé — onze composants à provisionner et faire tourner en continu (Keycloak, Kafka, N PostgreSQL, MinIO, Novu, OpenSearch, Redis, gateway).	Faible — une application, une base, Redis, MinIO.	Mo Re ser
Maintenance	Lourde — dépendances de version à synchroniser sur onze systèmes, chacun avec son cycle de mises à jour de sécurité.	Légère — un seul système à maintenir à jour.	Lég au
Disponibilité	Une panne d'un composant partagé (Kafka, Keycloak) peut dégrader plusieurs services à la	Panne = l'application entière est affectée, mais un seul système à surveiller et faire redémarrer.	La No mé

CRITÈRE	A — MICROSERVICES FASTAPI	B — DJANGO MODULAIRE	C — DJANGO MONOLITHIQUE
	fois — nouveau point de fragilité central, à haute disponibilité lui-même.		par le code.
Tolérance aux pannes	Bonne en théorie (isolation des pannes) — à condition d'implémenter correctement retries, circuit breakers et compensation, ce qui est un chantier en soi.	Limitée par nature — un bug grave affecte tout, mais le périmètre à tester avant déploiement est unique et bien connu.	Bonne en théorie — évènements sont vu.
Sécurité	Surface d'attaque étendue (onze systèmes exposés), mais Keycloak centralise correctement l'authentification si bien opéré.	Surface réduite, auth mature (Django + DRF/OAuth toolkit) — moins de composants à durcir.	Surface d'attaque large.
Facilité de déploiement	Difficile sans orchestrateur mature (Kubernetes ou équivalent) et pipelines par service — irréaliste sans équipe DevOps dédiée.	Simple — un pipeline, un artefact à déployer.	Simple — un pipeline pour pip.
Observabilité	Nécessite du tracing distribué (ex. OpenTelemetry) dès le premier jour pour rester compréhensible — sinon, diagnostiquer un incident devient très coûteux en temps.	Logs et métriques d'un seul processus — largement plus simple à observer et déboguer.	Preuve d'usage.
Transactions entre services	Pas de transaction ACID inter-services — nécessite le patron Saga (transactions compensatoires), complexe à concevoir et à tester correctement.	Transactions ACID natives — publier une pièce, l'enlever, la payer reste une seule transaction cohérente.	ACID (par les bases de données).
Cohérence des données	Cohérence à terme (eventual consistency) partout par défaut — à documenter et assumer pour chaque parcours utilisateur.	Cohérence forte partout, par défaut.	Cohérence forte partout, par défaut.

CRITÈRE	A — MICROSERVICES FASTAPI	B — DJANGO MODULAIRE	C —
			l'ac lég
Gestion des événements	Native et bien pensée (Kafka) — mais un bus d'événements à cette échelle est un système à part entière à administrer.	Signaux Django ou tâches Celery — suffisant pour un monolithe, pas conçu pour du inter-services à grande échelle.	Re au seu le j pay
Difficulté DevOps	Très élevée — orchestration multi-services, multi-bases, multi-pipelines ; suppose une équipe SRE/Platform dédiée.	Faible — proche de ce qui existe déjà sur Sendra aujourd'hui (deux applications Laravel).	Mo loir
Capacité 100k+ utilisateurs	Oui, mais la capacité vient de la bonne conception (cache, index, files d'attente), pas du découpage en services en soi — un monolithe mal indexé ne va pas plus loin en microservices.	Oui — de nombreuses plateformes de cette taille et bien au-delà tournent sur un monolithe correctement mis à l'échelle (cache, réplique de lecture, CDN).	Ou deu ser ma

PHASE 1

Socle, avant tout
développement métier

Le monolithe modulaire

- **Django**

avec
une
app
par
domaine
(users,
annonces,
pieces,
enlevements,
paiements,
admin,
statistiques)

—

frontières
nettes
dès
la
conception,
même
dans
une
seule
base.

- **MinIO**

pour
tous
les
fichiers,
dès
le
premier
commit

—

aucun
fichier
ne
touche
jamais
le
disque
local
d'un
serveur
applicatif.

- **Redis + Celery**

pour
l'asynchrone
(envois,
traitements
différables)
et
le
cache.

- **CI/CD + observabilité de base**

(le
chantier
identifié
comme
prioritaire
sur
Sendra
actuel,
valable
ici
aussi).

PHASE 2

Dès que le cœur métier
est stable

Les deux extractions sûres

- **Service Notifications**

,
consommant

les
événements
via
Redis
Streams
—
Novu
derrière,
pour
les
templates
et
canaux.

- **Service Recherche**

,
OpenSearch
alimenté
par
synchronisation
événementielle
depuis
le
cœur
—
jamais
source
de
vérité.

PHASE 3

Seulement si un signal
concret l'exige

Évolutions conditionnelles

- **Kafka**

si
le
volume
ou
la
durabilité
des

événements
dépasse
ce
que
Redis
Streams
tient
confortablement.

- **Keycloak**

si
un
vrai
besoin
de
SSO
multi-
partenaires
apparaît
(plusieurs
apps
distinctes,
clients
externes)
—
pas
par
défaut
pour
un
seul
écosystème
que
vous
contrôlez.

- **Extraction d'un troisième domaine**

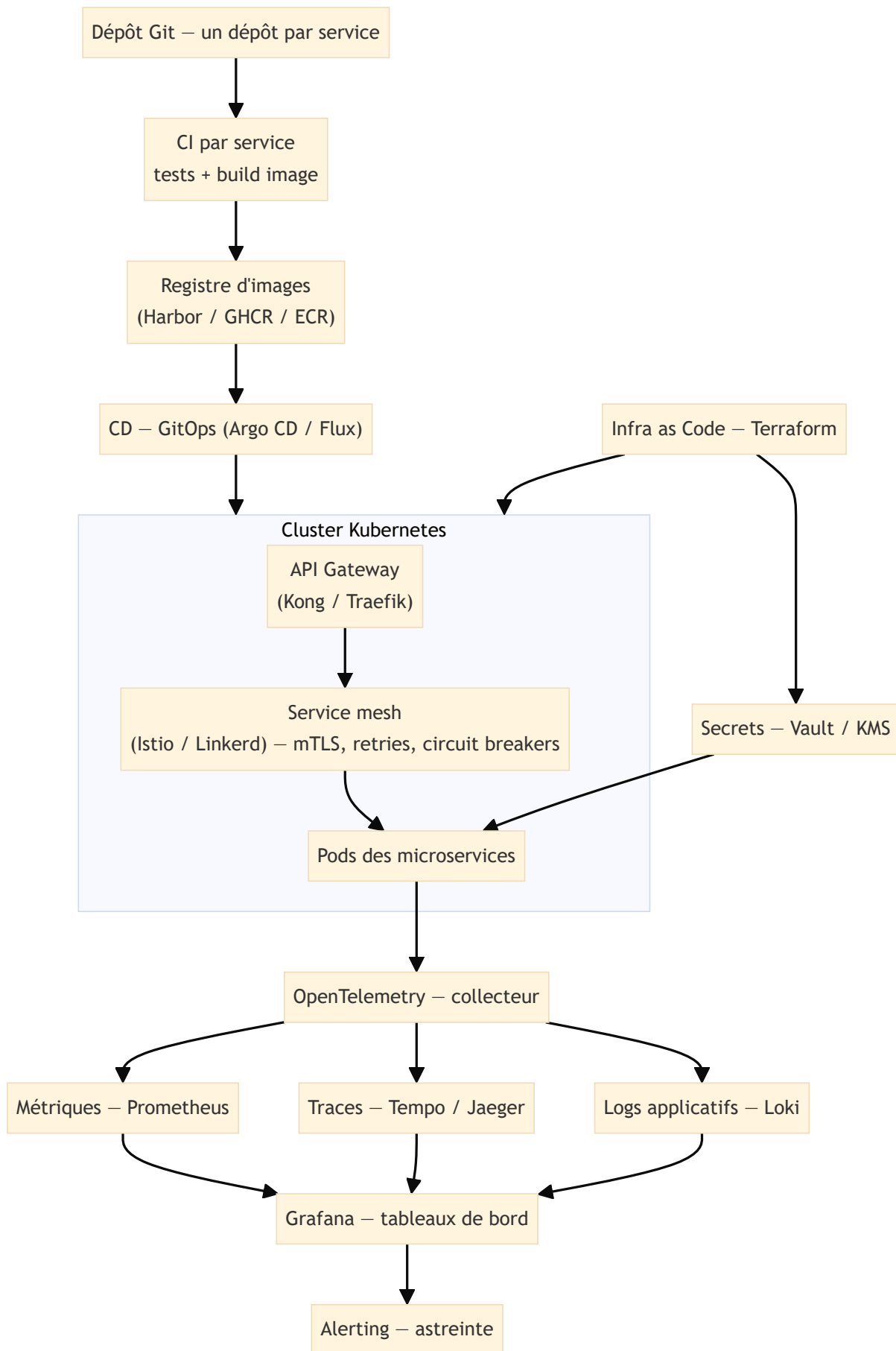
—
uniquement
si
une

équipe
dédiée
doit
le
faire
évoluer
à
un
rythme
propre,
jamais
par
principe.

11 La couche DevOps du Scénario A

Onze systèmes distincts ne tiennent que si l'infrastructure qui les relie est traitée comme un produit à part entière, pas comme un à-côté. Cinq briques, dans l'ordre où les monter.

PLATEFORME COMPLÈTE – ORCHESTRATION, RÉSEAU INTER-SERVICES, SECRETS, OBSERVABILITÉ



ORCHESTRATION

Kubernetes + GitOps

- Un cluster, un namespace par service — isolation des ressources et des droits.
- Argo CD ou Flux : l'état désiré vit dans Git, le cluster converge automatiquement — aucun déploiement manuel en SSH.
- Autoscaling horizontal par service, selon sa charge propre — le vrai bénéfice du découpage complet.

RÉSEAU INTER-SERVICES

Gateway + service mesh

- Un point d'entrée unique (API Gateway) pour les clients — routage, rate limiting, authentification des requêtes entrantes.
- Un mesh (Istio/Linkerd) pour le trafic *entre* services — mTLS automatique, retries, circuit breakers sans code applicatif dédié.

SECRETS & CONFIGURATION

Vault + Terraform

- Aucun mot de passe/clé en clair dans un `.env` commité — les onze systèmes du Scénario A en génèrent beaucoup.
- L'infrastructure elle-même versionnée (Terraform) — un cluster reconstructible, pas un serveur bricolé à la main.

OBSERVABILITÉ

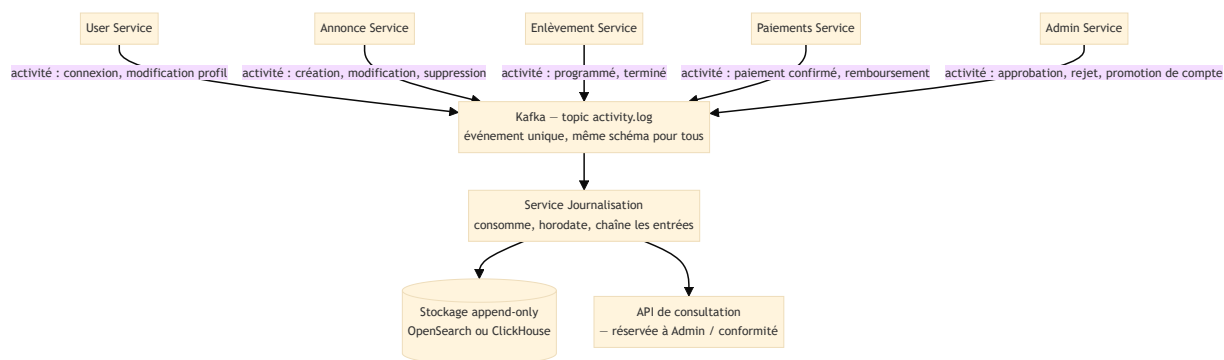
Métriques, traces, logs

- OpenTelemetry dès le premier service — instrumenter après coup, sur onze services, coûte largement plus cher.
- Le tracing distribué est ici la brique *non négociable* : sans lui, un parcours qui traverse quatre services devient impossible à déboguer.

12 Service de journalisation des activités

Distinct des logs techniques (erreurs, performance, déjà couverts par Loki) — celui-ci répond à une question métier : *qui a fait quoi, sur quoi, et quand*, à travers tous les services. Indispensable dès qu'il y a des paiements, des approbations, et plusieurs rôles avec des droits différents.

CHAQUE SERVICE PUBLIE SES ACTIVITÉS ; UN SEUL SERVICE EN GARDE LA TRACE



Ce qui rend ce service fiable, pas juste présent : un schéma d'événement unique et obligatoire pour tous les producteurs (acteur, action, ressource, horodatage, contexte) ; un stockage *append-only* où aucune entrée n'est jamais modifiée ni supprimée, seulement ajoutée ; et un accès en lecture strictement réservé (Admin, conformité) — la journalisation elle-même ne doit pas devenir une nouvelle fuite de données personnelles.

13 Autres services à considérer

Au-delà de la liste initiale, six services que le métier de Sendra (déconstruction, pièces recyclées, enlèvements) justifie concrètement — pas des services génériques de catalogue e-commerce.

TRAÇABILITÉ

Service Inventaire & Provenance

- Relie chaque pièce au véhicule dont elle est issue (VIN, date d'extraction, état).
- Distinct du Service Pièces (qui gère l'annonce commerciale) — celui-ci gère le stock physique et sa traçabilité réglementaire.

CONFIANCE

Service Avis & Réputation

- Notes et avis entre acheteurs et vendeurs de pièces — signal de confiance central dans une place de marché.
- Alimente le Service Recherche (classement) sans lui appartenir.

ANTI-FRAUDE

Service Risque & Fraude

- Consomme les événements Paiements + Annonces pour détecter annonces suspectes, comptes à comportement anormal, contestations de paiement.
- Un candidat naturel au machine learning une fois assez de volume/historique accumulé dans le Service Journalisation.

GÉOSPATIAL

Service Cartographie

- Formalise ce qui existe déjà dans Sendra actuel — recherche de pièces/enlèvements à proximité, PostGIS dédié.
- Se nourrit des mêmes événements que le

DOCUMENTS

Service Génération de documents

- PV de constatation, factures, attestations d'enlèvement — rendu PDF isolé, avec sa propre charge (pics lors des exports en masse).
- Lit les données via API, n'écrit jamais dans les

CONFIGURATION

Service de configuration & feature flags

- Active/désactive une fonctionnalité par service sans redéploiement — précieux dès onze services à faire évoluer indépendamment.

Service Recherche, avec un moteur spécialisé plutôt que OpenSearch seul.

bases des autres services.

- Permet aussi les déploiements progressifs (canary) service par service.

Le principe `1 service = 1 base = 1 déploiement` est juste en théorie et coûteux en pratique tant qu'aucun domaine n'a démontré un besoin réel d'indépendance — c'est la réserve que porte ce document. La décision de le maintenir malgré tout change ce qui suit : la couche DevOps (§11) et la journalisation des activités (§12) ne sont plus optionnelles, elles deviennent la condition pour que le Scénario A reste opérable.